扫描线与离线知识复习

对于一些问题而言，可能具有一些特点：虽然有修改和查询，但是不需要按照输入顺序实时的执行操作和完成查询，可以预先读入所有操作，并重新安排处理顺序，即 **离线处理** ，例如：

- 只有询问，没有修改操作
- 虽然有操作和询问，但是询问在所有操作执行完之后

例子

问题1. 给定一个序列，每次在一个区间 $[left, right]$ 内添加一个数字 x，求每个位置上最小的众数。

问题2. 给定一棵树，每次询问一条路径上 [a,b] 区间内的和。

问题3. 给定一个序列，每次询问一个区间 [left,right] 内比 x 小的数的和。

问题4. 给定若干个矩形，询问每个矩形覆盖的总面积/总周长。

问题5. 平面上有一些点，每次询问一个矩形内的点的数量。

问题6. 给定若干个点，询问一个给定长和宽能框住的最多的点的数量。

问题7. tarjan 方式求解 LCA 。

问题1

给定一个序列，每次在一个区间 $[left, right]$ 内添加一个数字 x，求每个位置上最小的众数。

步骤1. 差分转化： 对于每个操作 $[l, r, x]$ ，转化为 $\{l, x\}$ 表示 l 位置新增数 x ， $\{r+1, -x\}$ 表示 $r+1$ 位置上移除数 x 。对于每个位置 i ，我们记录 $events_i$ 表示 i 位置上的操作。

步骤2. 扫描线： 我们从左到右扫描，使用权值线段树维护当前范围内最小的众数。 对于每个位置 i ，执行完 $events_i$ 后，权值线段树的根节点就存储了该位置的答案。

具体维护方式：线段树节点需维护 max_cnt（最大出现次数）和 val（对应的数值）。在 PushUp 合并时，如果左右子树 max_cnt 相等，取 val 较小的那个（以满足“最小众数”的要求）。

步骤3. 输出答案： 我们输出每个位置上的答案。

操作没有按照给定顺序执行，操作被离线处理。

问题2

给定一棵树，每次询问一条路径上点权在 [a,b] 区间内的和。

步骤1. 差分转化： 对于每个查询 $[x, y, a, b]$ ，转化为 $\{x, y, a-1\}$ 表示路径 $x \sim y$ 上权值在 $[1, a-1]$ 区间的总和。同理，$\{x, y, b\}$ 表示路径 $x \sim y$ 上权值在 $[1, b]$ 区间的总和。

步骤2. 点权离线加入： 处理 $x, y, 1$ 的问题时，显然不需要大于 1 的点权，因此点权需要按值的顺序依次加入。

步骤3. 排序: 对点权和询问分别按照权值从小到大排序。

步骤4. 双指针解决询问： 维护两个指针 i, j ，表示当前处理到的询问编号和点的下标（排序后的下标）。如果当前点 j 权值小于询问 $i.val$ ，那么将点插入线段树，并将指针 j 向右移动，直到 j 指向的点权值大于 $i.val$ 。此时即可结合树链剖分回答当前询问。

步骤5. 输出答案： 我们输出每个位置上的答案。

询问没有按照给定顺序回答。

问题3

给定一个序列，每次询问一个区间 [left,right] 内比 x 小的数的和。

如何解决?

按照前面的经验，是离线询问区间，还是离线权值 x 呢?

步骤1. 差分转化： 询问$\{l, r, x\}$ 转化为 $\{1, l-1, x\}$ 和 $\{1, r, x\}$ 。

步骤2. 排序： 对于相同的前缀，求解答案需要的是同一棵权值线段树。因此，按照前缀的结束位置排序。(或者将拆分的询问，全部挂在结束位置上)
即

```
for(int i=1;i<=m;i++){
    int l,r,x;
    cin>>l>>r>>x;
    qs[l].push_back({x,1,i});
    qs[r+1].push_back({x,-1,i});
}
```

步骤3. 扫描线：扫描序列，插入数字，同时解决对应前缀的答案。

```
for (int i = 1; i <= n; i++) {
    update(1, 1, m, a[i], a[i]);
    for (auto &q : qs[i]) {
        long long val = query(1, 1, m, 1, q.x);
        if (q.sign == 1) ans[q.id] += val;
        else ans[q.id] -= val;
    }
}
```

问题4

给定若干个矩形，询问每个矩形覆盖的总面积/总周长。

面积

步骤1. 差分转化： 每个矩形$\{x_1, y_1, x_2, y_2\}$拆分为入边 $\{x_1, x_2, y_1, +1\}$和出边 $\{x_1, x_2, y_2, -1\}$ 。

步骤2. 排序：总面积的本质是覆盖的边长 * 高，高就是相邻两条边的 y 差值。

步骤3. 扫描线：扫描线沿y轴向上移动，维护当前 x 轴上有矩形覆盖的长度。在插入相同 y 坐标的所有边之后，查询根节点，计算答案即可。

细节：
线段树节点需要维护 cnt（该区间被完全覆盖的次数）和 len（该区间内被覆盖的有效长度）。

如果 cnt > 0，则 len = 区间总长。

如果 cnt == 0，则 len = left_child.len + right_child.len（叶子节点为0）。

周长

与面积相似，区别在于：

- 在不同的 y 坐标处，边长的贡献为 与上次的差值。
- 水平方向与竖直方向分别考虑。

问题5

步骤1. 差分转化： 将矩形询问 (x_1, y_1, x_2, y_2) 拆分为二维前缀和形式：
$Sum(x_2, y_2) - Sum(x_1 - 1, y_2) - Sum(x_2, y_1 - 1) + Sum(x_1 - 1, y_1 - 1)$。

步骤2. 排序： 将所有点和询问按 y 排序。

步骤3. 扫描线：扫描 y，用线段树维护 x 轴上各个区间内的点数。

问题6

思考：如何解决?

对于点 $\{x, y\}$ 而言，能将其框住的矩形 (W, H) 的左下角 (A, B) 范围为：

$A \leq x \leq A + W - 1$

$B \leq y \leq B + H - 1$

这个范围也是一个矩形。

只要矩形的左下角放在这个范围内，都可以框住一个点，即贡献 + 1。

那么，原问题可以转化为：

给定若干个矩形，每个矩形都有一个权值+1，求某个位置最大的点权。

==> 给出若干张地毯，求被盖住最多次的一个位置，输出这个位置。

如何扫描线?

步骤1. 差分转化： 对于每个地毯 $\{x_1, y_1, x_2, y_2\}$ ，转化为 $\{x_1, x_2, y_1, +1\}$ 和 $\{x_1, x_2, y_2 + 1, -1\}$ 。

- 注意实现，y_2 处的地毯需要在 $y_2 + 1$ 处撤销。

步骤2. 排序： 按照 y 坐标排序。

步骤3. 扫描线： 扫描线维护当前 x 轴上每个位置被覆盖的次数。在插入相同 y 坐标的所有地毯之后，查询根节点，计算答案即可。

- 即 y_1 处，将 x_1 到 x_2 之间的所有位置权值 +1。
- 在 $y_2 + 1$ 处，将 x_1 到 x_2 之间的所有位置权值 -1。

```
for (auto line : lines) {
    update(1, 1, max_val, line.x1, line.x2, line.type);
    ans = max(ans, query(1, 1, max_val, 1, max_val));
}
```

变式

Vjudge T2 如何解决?

问题7

LCA

核心点： 在 DFS 过程中，总会先经过 x ，再经过 lca（y 所在子树的根节点），然后到达 y 。

基于此，就可以一次 DFS 解决所有 LCA 问题。

实现

- 在 DFS 回溯时，将子节点 v 的集合合并到父节点 u 上。
- 当访问到节点 u 时，处理所有与 u 相关的查询 (u, v)。
  - 如果 v 已经被访问过（且已回溯处理完毕），则 $LCA(u, v) = Find(v)$（即 v 所在集合的代表元素，也就是当前的祖先）。

```
void tarjan(int u) {
    vis[u] = true; // 标记 u 已访问
    for (int v : adj[u]) {
        if (!vis[v]) {
            tarjan(v);
            fa[v] = u;
        }
    }
    for (auto &q : queries[u]) {
        int v = q.first;
        int id = q.second;
        if (vis[v]) ans[id] = find(v);
    }
}
```